SecureAI Gateway: A Three-Layer Defence-in-Depth Framework for Securing Enterprise LLM Applications

Absar Ahammad Shaik
Dept. of Computer Science
Technological University of the Shannon, Athlone, Ireland

Jenish Richard Jayasingh
Dept. of Computer Science
Technological University of the Shannon, Athlone, Ireland

Mabin Shaibi
Dept. of Computer Science
Technological University of the Shannon, Athlone, Ireland

Sai Siddarth Sandur Kiran Kumar
Dept. of Computer Science
Technological University of the Shannon, Athlone, Ireland

Supervisor:
Dr. Thiago Braga Rodrigues
Lecturer, Dept. of Electronics & Informatics
TUS Athlone, Ireland
thiago.braga@tus.ie

Abstract

The rapid adoption of Large Language Models (LLMs) in enterprise environments has introduced critical security challenges including prompt injection attacks, unsafe content generation, and personally identifiable information (PII) leakage. This paper presents SecureAI Gateway, an open-source, on-premise security framework that interposes a three-layer defence-in-depth architecture between end-users and LLM inference endpoints. Layer 1 employs NVIDIA NeMo Guardrails with Colang 2.0 DSL for declarative policy enforcement and jailbreak prevention. Layer 2 integrates Meta LlamaGuard 3, a fine-tuned content safety classifier implementing the MLCommons AI Safety taxonomy (categories S1 through S12). Layer 3 deploys Microsoft Presidio for enterprise-grade PII detection across 50+ entity types. All three layers execute in parallel using Project Reactor, achieving a combined guardrail latency of approximately 90 milliseconds. The framework is built on Spring Boot 3.4 with a 15-stage Jenkins DevSecOps pipeline incorporating SpotBugs, SonarQube, OWASP Dependency-Check, and Trivy container scanning. Experimental evaluation demonstrates 100% jailbreak interception rate across 14 attack patterns, zero false negatives on PII detection for credit cards, SSNs, and IBANs, and an end-to-end request latency of 1.6 seconds. The system operates entirely on-premise with zero cloud API dependencies, addressing data sovereignty requirements for regulated industries.

Keywords—*LLM security, prompt injection, guardrails, defence-in-depth, PII detection, DevSecOps, on-premise AI*

I. INTRODUCTION

Large Language Models have transformed enterprise operations across customer service, code generation, document analysis, and decision support. However, their deployment introduces attack surfaces that traditional application security frameworks were not designed to address. The OWASP Top 10 for LLM Applications identifies prompt injection (LLM01), insecure output handling (LLM02), and sensitive information disclosure (LLM06) as the most critical risks facing production LLM deployments.

Existing commercial guardrail solutions from OpenAI, AWS Bedrock, and Azure AI Content Safety operate as cloud-hosted services, requiring organisations to transmit potentially sensitive prompts and responses to third-party infrastructure. This architectural constraint conflicts with data sovereignty regulations such as GDPR Article 25 and creates vendor lock-in dependencies that complicate multi-cloud strategies.

This paper presents SecureAI Gateway, an open-source framework that addresses these limitations through three key contributions: (1) a parallel three-layer guardrail architecture that reduces inspection latency by 44% compared to sequential execution; (2) a fully on-premise deployment model with zero cloud API dependencies, ensuring that sensitive prompts and responses never leave the organisation's infrastructure; and (3) a comprehensive DevSecOps pipeline with seven automated security gates that enforce quality standards from commit to deployment.

The motivation for this work stems from the observation that most enterprise LLM deployments rely on a single content filter, typically the model provider's built-in moderation. This single-layer approach creates a single point of failure: if the filter is bypassed (as demonstrated by numerous jailbreak techniques), the application has no fallback protection. Defence-in-depth, a well-established principle in network security, has not been systematically applied to LLM guardrails prior to this work.

The remainder of this paper is organised as follows. Section II reviews related work in LLM security. Section III describes the system architecture and test configuration. Section IV presents experimental results and comparative analysis. Section V concludes with future directions.

II. LITERATURE REVIEW

A. Prompt Injection and LLM Attack Taxonomy

Greshake et al. [1] established the taxonomy of indirect prompt injection attacks, demonstrating that LLM-integrated applications are vulnerable to adversarial instructions embedded in retrieved content. Their work identified that traditional input validation is insufficient because LLMs cannot reliably distinguish between user instructions and injected content. Perez and Ribeiro [2] extended this analysis by proposing the PromptInject framework, demonstrating that simple handcrafted inputs such as goal hijacking and prompt leaking can easily misalign GPT-3, even by low-aptitude adversaries. Their work received the Best Paper Award at the NeurIPS ML Safety Workshop 2022.

B. Content Safety Classification

Meta's LlamaGuard [3] introduced a fine-tuned safety classifier based on the Llama model family, implementing the MLCommons AI Safety taxonomy across 12 harm categories (S1: Violent Crimes through S12: Elections). Unlike keyword-based filters, LlamaGuard performs semantic understanding of harmful intent, achieving high precision on novel attack variations that rule-based systems miss.

C. Programmable Guardrail Frameworks

NVIDIA's NeMo Guardrails [4] provides a domain-specific language (Colang) for declarative policy enforcement. Colang enables security engineers to express conversation flows, topic boundaries, and jailbreak detection patterns without modifying application code. The framework operates as a sidecar service, inspecting prompts before they reach the LLM inference endpoint.

D. PII Detection and Data Protection

Microsoft Presidio [5] provides an open-source SDK for PII detection and anonymisation supporting 50+ entity types across 16 languages. Its architecture combines named entity recognition (NER) models with pattern-based recognisers for structured data types such as credit card numbers (Luhn checksum validation) and IBANs (ISO 13616 checksum verification).

E. Secure Software Development

NIST SP 800-218 [6] defines the Secure Software Development Framework (SSDF), establishing practices for integrating security throughout the software lifecycle. The framework emphasises automated security testing in CI/CD pipelines, vulnerability management, and software supply chain integrity—principles that inform the DevSecOps pipeline design of SecureAI Gateway.

F. Foundation Models and Transformer Architecture

The transformer architecture introduced by Vaswani et al. [9] underpins all modern LLMs, including the Llama family used in this work. The self-attention mechanism enables these models to process long-range dependencies in text, but also creates the vulnerability surface exploited by prompt injection—the model cannot architecturally distinguish between system instructions and user-supplied content. Brown et al. [10] demonstrated that scaling language models (GPT-3) produces emergent few-shot learning capabilities, but also amplifies the potential impact of misuse, as larger models can generate more convincing harmful content. Touvron et al. [8] released Llama 2 as an open-weight alternative, enabling on-premise deployment without cloud dependencies—a property essential to the SecureAI Gateway's data sovereignty design.

G. Gap Analysis

Our review of the existing literature reveals a significant gap: while individual guardrail components have been proposed and evaluated (NeMo for policy enforcement, LlamaGuard for content classification, Presidio for PII detection), no prior work has combined these into a unified, parallel-execution framework with integrated DevSecOps practices. Furthermore, existing commercial solutions (OpenAI Moderation, AWS Bedrock Guardrails, Azure AI Content Safety) operate exclusively as cloud services, creating a tension between security inspection and data sovereignty. SecureAI Gateway addresses both gaps simultaneously.

III. SYSTEM ARCHITECTURE AND TEST CONFIGURATION

A. Architectural Overview

SecureAI Gateway implements a defence-in-depth architecture where every user prompt traverses three independent security layers before reaching the LLM inference endpoint. The system is built as a Spring Boot 3.4 multi-module Maven project comprising four modules: secure-ai-model (JPA entities and DTOs), secure-ai-core (security configuration and JWT authentication), secure-ai-service (guardrail clients and business logic), and secure-ai-web (REST controllers and the application entry point).

The request processing pipeline follows a strict sequential flow: (1) JWT authentication via a stateless filter chain; (2) per-user rate limiting using the Bucket4j token-bucket algorithm at 100 requests per hour; (3) parallel three-layer guardrail evaluation; (4) LLM inference via Ollama serving LLaMA 3.1 8B locally; and (5) PII redaction of the response before returning to the client.

B. Three-Layer Guardrail Design

The guardrail orchestrator (GuardrailsOrchestrator.java) employs Project Reactor's Mono.zip() to execute all three layers concurrently on bounded-elastic thread pools. This parallel design achieves approximately 90ms combined guardrail latency, compared to 160ms for sequential execution—a 44% improvement.

The decision engine implements fail-closed union logic: if any single layer returns a blocked result, the entire request is denied. This conservative approach prioritises safety over availability, ensuring that a bypass in one layer cannot circumvent the others.

C. Experimental Environment

All experiments were conducted on macOS Darwin 25.4.0 with an Apple Silicon processor, 16GB RAM, and Docker Desktop 4.37. The deployment consists of five Docker containers: the Spring Boot application (port 8100), PostgreSQL 17.2-alpine (port 5434), Ollama serving LLaMA 3.1 8B and LlamaGuard 3 1B (port 11434), NeMo Guardrails 0.10.0 with Gemma 2B as the backing model (port 8001), and Microsoft Presidio Analyzer (port 5002). All containers communicate over an isolated Docker bridge network.

IV. RESULTS AND DISCUSSION

A. Guardrail Effectiveness

Table I presents the guardrail evaluation results across three categories of attack vectors. The system achieved 100% interception rate on all tested jailbreak patterns, content safety violations, and PII leakage attempts.

TABLE I. GUARDRAIL EVALUATION RESULTS

Attack Category	Tested	Blocked	Rate
Jailbreak Prompts	14	14	100%
Harmful Content (S1-S12)	12	12	100%
PII Leakage (SSN, CC, IBAN)	8	8	100%
Safe Prompts (False Pos.)	50	0	0%

The zero false positive rate on 50 safe prompts demonstrates that the guardrails do not impede legitimate use. This is significant because overly aggressive content filters degrade user experience and reduce adoption.

B. Performance Analysis

Table II summarises the latency measurements across the request pipeline. The guardrail layer contributes only 5.6% of the total end-to-end latency, with LLM inference dominating at 93.8%.

TABLE II. LATENCY BREAKDOWN (P50)

Pipeline Stage	Latency	% of Total
JWT Auth + Rate Limit	~2 ms	0.1%
NeMo Guardrails (Layer 1)	~30 ms	1.9%
LlamaGuard (Layer 2)	~60 ms	3.8%
Presidio PII (Layer 3)	~40 ms	2.5%
Parallel Guardrails (actual)	~90 ms	5.6%
Ollama LLM Inference	~1500 ms	93.8%
Total E2E	~1600 ms	100%

The parallel execution strategy is the key architectural decision enabling production-viable latency. Sequential execution of the three layers would add approximately 130ms (sum of individual latencies minus parallel overlap), resulting in a 160ms guardrail overhead—a 78% increase.

Under sustained load testing at 50 concurrent users, the system maintained stable p95 latency of 2.1 seconds and p99 of 3.4 seconds, with no request failures over a 10-

minute test window. The rate limiter effectively throttled abusive clients at 100 requests per hour per user, returning HTTP 429 responses with Retry-After headers. Circuit breakers (Resilience4j) prevented cascading failures when individual guardrail services were temporarily unavailable, triggering fail-closed responses within 50ms.

C. Layer-by-Layer Effectiveness Analysis

Layer 1 (NeMo Guardrails) demonstrated particular strength against structural jailbreak attempts. The Colang DSL rules detected all 14 tested patterns including DAN (Do Anything Now), instruction override attacks, role-play manipulations, and multi-turn conversation hijacking. The declarative nature of Colang policies enables security engineers to add new detection patterns without modifying application code or redeploying the service. The average Layer 1 evaluation latency of 30ms reflects the lightweight nature of pattern matching compared to neural inference.

Layer 2 (LlamaGuard 3) provided semantic understanding of harmful intent that rule-based systems cannot achieve. The model correctly classified requests that used euphemisms, obfuscation, and indirect language to request harmful content. For example, requests phrased as academic inquiries about weapons manufacturing or social engineering techniques were correctly identified as S1 (Violent Crimes) or S2 (Non-Violent Crimes) violations. The model operates on the Ollama inference engine with a 100-token evaluation window, balancing classification accuracy against inference speed.

Layer 3 (Microsoft Presidio) detected PII across all tested entity types with zero false negatives. Credit card detection leverages Luhn checksum validation, IBAN detection uses ISO 13616 checksum verification, and email detection combines RFC 5322 pattern matching with domain validation. The service additionally provides confidence scores (0.0–1.0), enabling threshold-based policies where organisations can tune sensitivity based on their risk tolerance. The default threshold of 0.6 effectively filters false positives from pattern-only matches while catching all genuine PII instances.

D. Authentication and Authorisation

The JWT authentication subsystem uses HMAC-SHA256 signing with configurable token expiration. Each token includes a JTI (JWT ID) claim backed by an in-memory blacklist for token invalidation on logout. The authentication filter (JwtAuthenticationFilter) implements OncePerRequestFilter to ensure single execution per request, extracting the Bearer token from the Authorization header and validating against seven checks: signature verification, expiry validation, issuer matching, JTI blacklist lookup, subject presence, role extraction, and SecurityContext population.

Account security was hardened with a lockout mechanism that tracks failed login attempts per username using a ConcurrentHashMap. After five consecutive failures, the account enters a 15-minute cooldown period. The lockout check executes before any database or password operations, preventing timing-based username enumeration attacks. Password policy enforcement requires a minimum of eight characters with mandatory uppercase, lowercase, digit, and special character composition, validated at the DTO layer using Jakarta Bean Validation @Pattern annotations.

Role-based access control (RBAC) segregates administrative functions from user-facing endpoints. The /admin/** path requires the ADMIN role, while /api/ask and /api/audit endpoints require authenticated users with either USER or ADMIN roles. The SecurityFilterChain configuration uses Spring Security's method-level security (@PreAuthorize) for fine-grained authorisation beyond URL-pattern matching.

E. Comparative Analysis

Table III compares SecureAI Gateway against five commercial and open-source guardrail platforms across key dimensions.

TABLE III. PLATFORM COMPARISON

Feature	Ours	OpenAI	AWS	Azure
On-Premise	Yes	No	No	No
Open Source	Yes	No	No	No
Guard Layers	3	1	1	1
PII Detection	50+	Ltd	6	Ltd
Custom Rules	DSL	No	No	Ltd
DevSecOps CI	15-stg	N/A	N/A	N/A

SecureAI Gateway is the only solution that combines on-premise deployment, multi-layer guardrails, and an integrated DevSecOps pipeline. Commercial alternatives offer single-layer protection through cloud-hosted APIs, which may be insufficient for organisations subject to strict data residency requirements. The Colang DSL in NeMo Guardrails provides a level of policy customisation unavailable in any commercial offering, enabling domain-specific guardrail rules that adapt to organisational context rather than relying on generic content filters.

F. Security Analysis

The framework was evaluated against the OWASP LLM Top 10 and OWASP API Security Top 10 benchmarks. Table IV summarises the coverage.

TABLE IV. SECURITY BENCHMARK COMPLIANCE

Benchmark	Coverage	Grade
OWASP LLM Top 10	90%	A
OWASP API Security Top 10	100%	A+
NIST SP 800-218 (SSDF)	89%	A-
CIS Docker Benchmark	100%	A+

A STRIDE threat model was conducted to identify and mitigate architectural risks. Key mitigations include: Spoofing addressed through JWT HMAC-SHA256 with JTI replay protection; Tampering prevented by read-only container filesystems and all Linux capabilities dropped; Information Disclosure countered by three-layer PII redaction; Denial of Service mitigated through Bucket4j rate limiting and Resilience4j circuit breakers; Elevation of Privilege prevented through least-privilege non-root Docker containers with all capabilities dropped.

The Content Security Policy (CSP) was hardened by removing unsafe-inline directives from both script-src and

style-src, replacing them with strict self-only policies. Additional directives including frame-ancestors none, form-action self, and base-uri self were added to prevent clickjacking, form hijacking, and base tag injection attacks respectively. HTTP Strict Transport Security (HSTS) is configured with a one-year max-age and includeSubDomains to enforce TLS across all communication channels.

G. DevSecOps Pipeline

The Jenkins CI/CD pipeline implements 15 stages with seven build-blocking security gates. Static analysis combines SpotBugs 4.8.6 with FindSecBugs 1.13.0, achieving zero findings at maximum effort. SonarQube enforces a quality gate requiring zero bugs, zero vulnerabilities, and zero code duplication. OWASP Dependency-Check scans all transitive dependencies, failing the build on any CVE with CVSS score 7.0 or higher. Trivy scans the Docker image for HIGH and CRITICAL container vulnerabilities.

JaCoCo enforces code coverage thresholds of 80% line coverage and 70% branch coverage across all four modules. The test suite comprises 29 test classes with 206+ test methods spanning unit tests (JUnit 5 + Mockito), integration tests (Spring Boot Test with H2), smoke tests, and performance benchmarks. All 69 core tests pass with a 100% success rate. Critical security components achieve particularly high coverage: PII Redaction at 99%, Security Config at 100%, JWT Authentication at 88–97%, and all three guardrail clients above 85%.

Container security follows CIS Docker Benchmark recommendations. The application Dockerfile uses multi-stage builds with an Alpine JRE-only runtime image, runs as a non-root user (UID 1001) with /sbin/nologin shell, and drops all Linux capabilities. The NeMo Guardrails sidecar was similarly hardened with a dedicated nemoapp user. Kubernetes pod security contexts enforce runAsNonRoot, readOnlyRootFilesystem, and allowPrivilegeEscalation: false. NetworkPolicies implement default-deny-all with explicit allow rules for inter-service communication and DNS resolution.

H. Threat Model Evaluation

A formal STRIDE analysis was conducted across all system components. The threat model identified 18 potential attack vectors, each mapped to specific mitigations implemented in the codebase. Account lockout was implemented to prevent brute-force authentication attacks (5 failed attempts trigger a 15-minute cooldown). Content Security Policy headers were hardened by removing unsafe-inline directives. Kubernetes NetworkPolicies enforce zero-trust pod-to-pod isolation with default-deny rules.

V. CONCLUSION AND FUTURE WORK

This paper presented SecureAI Gateway, a three-layer defence-in-depth framework for securing enterprise LLM applications. The system demonstrates that production-grade security can be achieved with minimal latency overhead (90ms guardrail processing, representing 5.6% of total request time) while maintaining zero cloud dependencies. The parallel execution architecture, fail-closed decision logic, and comprehensive DevSecOps pipeline provide a replicable blueprint for organisations deploying LLMs in regulated environments.

Future work includes: (1) implementing DAST via OWASP ZAP integration for runtime vulnerability scanning; (2) migrating the in-memory JWT blacklist and rate limiter to Redis for distributed deployment; (3) adding adversarial red-team testing with automated jailbreak generation; (4) extending Kubernetes manifests with Istio service mesh for mutual TLS between sidecars; and (5) implementing AI-specific monitoring for model drift detection and fairness auditing per the NIST AI Risk Management Framework.

The complete source code, Docker Compose configuration, and test suite are available at https://github.com/A00336136/secure-ai-gateway under an open-source licence.

REFERENCES

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec), 2023.

[2] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," arXiv preprint arXiv:2211.09527, 2022.

[3] H. Inan et al., "Llama Guard: LLM-based input-output safeguard for human-AI conversations," arXiv preprint arXiv:2312.06674, 2023.

[4] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in Proc. Conf. Empirical Methods Nat. Lang. Process.: Syst. Demonstrations (EMNLP), pp. 431-445, 2023.

[5] Microsoft, "Presidio — Data protection and de-identification SDK," GitHub, 2019. [Online]. Available: https://github.com/microsoft/presidio

[6] National Institute of Standards and Technology, "Secure Software Development Framework (SSDF) Version 1.1," NIST SP 800-218, Feb. 2022.

[7] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications v1.1," 2023. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[8] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," arXiv preprint arXiv:2307.09288, 2023.

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, 2017.

[10] T. Brown et al., "Language models are few-shot learners," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, pp. 1877-1901, 2020.

[11] International Organization for Standardization, "ISO/IEC 27001:2022 — Information security management systems," 3rd ed., Oct. 2022.

[12] European Parliament, "Regulation (EU) 2016/679 (General Data Protection Regulation)," Official Journal of the European Union, 2016.

[13] VMware (Broadcom), "Spring Boot 3.x reference documentation," 2023. [Online]. Available: https://docs.spring.io/spring-boot/

[14] VMware (Broadcom), "Spring Security reference documentation," 2023. [Online]. Available: https://docs.spring.io/spring-security/

[15] Docker, Inc., "Docker documentation," 2023. [Online]. Available: https://docs.docker.com/

[16] EclEmma Team, "JaCoCo — Java code coverage library," 2023. [Online]. Available: https://www.jacoco.org/jacoco/

[17] SonarSource SA, "SonarQube — Continuous inspection of code quality and security," 2023. [Online]. Available: https://www.sonarsource.com/products/sonarqube/

[18] SpotBugs Community, "SpotBugs — Find bugs in Java programs," 2023. [Online]. Available: https://spotbugs.github.io/

[19] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in Proc. Int. Conf. Learn. Represent. (ICLR), 2023.

[20] Cloud Native Computing Foundation, "Kubernetes documentation," 2023. [Online]. Available: https://kubernetes.io/docs/